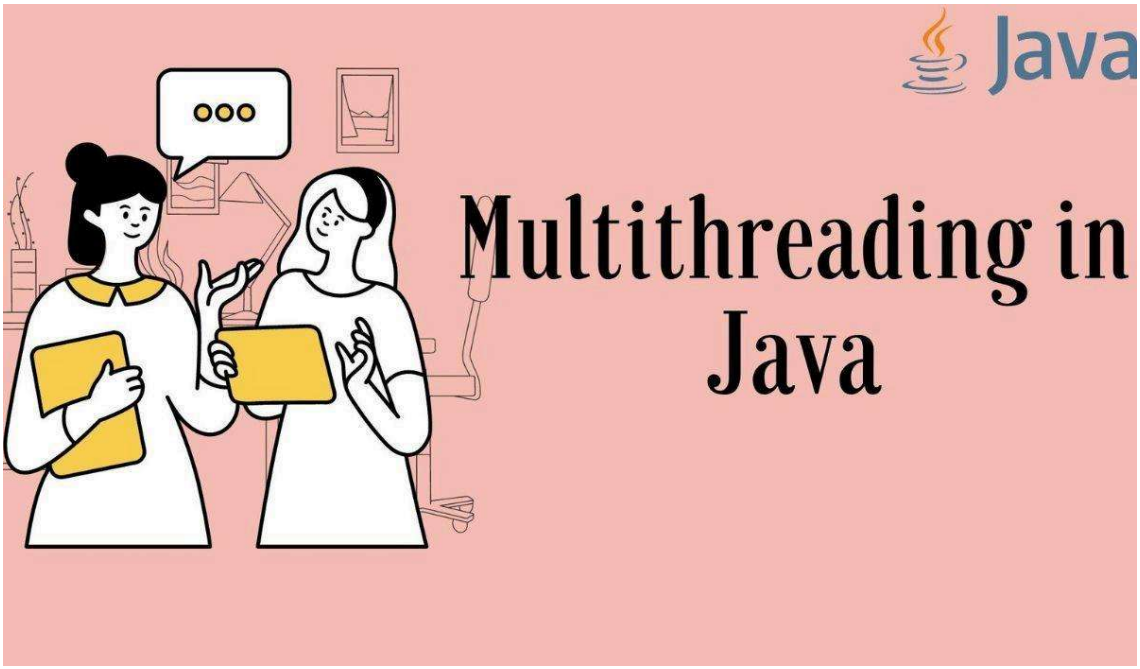
Java Multithreading



Multitasking

Multitasking means doing multiple tasks at the same time.
Example: Listening to music 🎵 while downloading a file 💾

A student is studying from a laptop, writing notes in a notebook, and also using a phone at the same time. This is called multitasking because the student is doing multiple tasks simultaneously.

- A student is listening to a lecture 🎧 and taking notes 📒 at the same time.
- A girl is scrolling social media 📱 while attending a lecture.
- A student is doing homework and thinking about food at the same time.

Multiprogramming, Multitasking, Multithreading, and Multiprocessing

1. Multiprogramming

Idea:
Many programs are kept in memory, but the CPU works on one at a time. When one program is waiting (like for I/O), the CPU switches to another.
Goal: Maximize CPU utilization
Example:
You open a browser, music player, and editor — but the CPU runs one, switches when needed.

2. Multitasking

Idea:
Running multiple tasks apparently simultaneously. CPU switches between tasks very fast (time-sharing). Gives an illusion of parallel work.
Example:
Listening to music while typing notes and downloading a file.
Difference from Multiprogramming:

- Multiprogramming = focus on CPU usage
- Multitasking = focus on user experience (doing many things at once)

3. Multithreading

Idea:
A single program has multiple threads (smaller units of execution). Threads share the same memory. Faster communication than separate programs.
Example:- A web browser:

- One thread loads the page
- One handles scrolling
- One plays video

4. Multiprocessing

Idea:
Using multiple CPUs/cores to run processes simultaneously. True parallel execution. Each process runs on a different core.
Example - Your system with 8 cores running:

- Game
- Browser
- Background tasks — all truly at the same time

Quick Comparison Table:

Concept	Focus	Parallelism	Example
Multiprogramming	CPU utilization	No (switches on I/O)	Browser + Music + Editor in memory
Multitasking	User experience	Apparent (time-sharing)	Music + Typing + Downloading
Multithreading	Single program, many tasks	Apparent / True (multi-core)	Browser: load + scroll + video
Multiprocessing	Multiple CPUs/cores	True parallel	Game + Browser + Background tasks

Multithreading

Multithreading means running multiple tasks simultaneously within a single program, like a student studying, using a phone, and attending a class at the same time. Multithreading is when a program performs multiple tasks simultaneously using threads.

📝 **Note:** Multithreading = many tasks at the same time in one program.

Thread in Java

A thread is a small part of a program that performs a single task.

Example: A student studying, using a phone, and listening to music.

- Studying = one thread
- Using phone = second thread
- Listening to music = third thread

📝 **Note:** Each activity is like a thread.

A thread is the smallest unit of a program that can run independently and allows a program to perform multiple tasks at the same time, where multiple threads can work together, sharing the same memory and communicating with each other.

Process vs Thread

Feature	Process	Thread
Definition	Independent program in execution	The smallest unit of a process
Memory	Has its own memory space	Shares memory with other threads

Communication	Inter-process communication (slow)	Direct shared memory (fast)
Creation overhead	Heavy (more resources)	Light (fewer resources)
Crash effect	One crash doesn't affect others	One crash can affect all threads
Example	Chrome browser = one process	Each browser tab = one thread

Ways to Create a Thread in Java:-

There are three ways to create a thread in Java:

Way 1: By Extending the Thread Class

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();
    }
}
```

Step-by-step Explanation:-

1. Create a new class that extends Thread
2. Override the run() method to define the task
3. Create an object of your class and call start() to run the thread

Way 2: By Implementing the Runnable Interface

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread is running");
    }
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyRunnable());
        t1.start();
```

```
    }
}
```

Step-by-step explanation:-

1. Create a class that implements Runnable
2. Override the run() method
3. Create a Thread object with your class object and call start()

📝 **Note:** start() begins a thread and calls run() internally.

Thread Lifecycle in Java

A thread in Java goes through different states from creation to termination. Java officially defines 6 states inside the Thread.

1: New (Created)

- The thread is created but not started yet
- Example: **Thread t = new Thread()**
  The thread is in a new state

2: Runnable

- When start() is called, the thread becomes ready to run
- The thread waits for the CPU scheduler to give it CPU time
- It is now in a runnable state

3: Running

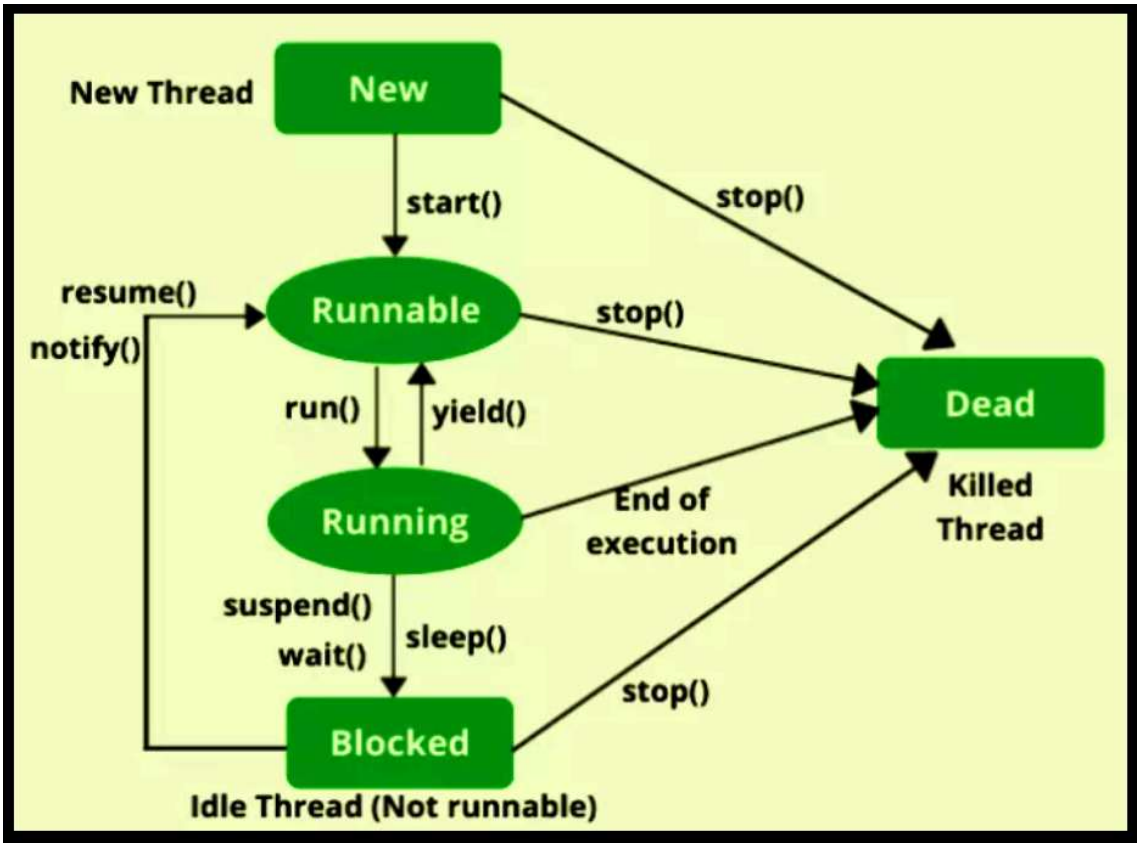
- When the CPU scheduler selects a thread from the runnable state, it executes the run() method
- The thread is now in a running state

4: Waiting/Blocked

- The thread can be temporarily inactive while waiting for resources or another thread
- Methods like sleep(), join(), wait() put a thread in waiting/blocking state

5: Terminated (Dead)

- When the thread completes execution, it moves to the terminated state
- It cannot be started again.



Thread Lifecycle Summary Table:

State	Description	How Thread Enters This State	Example / Methods
New (Created)	The thread is created but not started yet	When a Thread object is created	Thread t = new Thread();
Runnable	The thread is ready to run and waiting for CPU time	When the start() method is called	t.start();
Running	The thread is executing the run() method	When the CPU scheduler selects a thread from the runnable state	run() method executes

Waiting / Blocked	The thread is temporarily inactive, waiting for resources or another thread	When a thread waits for a time, a lock, or another thread	sleep(), wait(), join()
Terminated (Dead)	The thread has completed execution and cannot restart	When run() method finishes	Thread execution ends

Important Thread Methods

Method	Description
start()	Starts the thread; internally calls run()
run()	Contains the task the thread performs
sleep(ms)	Pauses thread for given milliseconds; does NOT release lock; causes TIMED_WAITING
yield()	Suggests CPU to give a chance to other threads of the same priority; not guaranteed
join()	Makes the calling thread wait until this thread finishes
isAlive()	Returns true if the thread is still running, false otherwise
setPriority(n)	Sets thread priority between 1 and 10
getPriority()	Returns the thread's current priority
getName()	Returns the thread's name

Thread Priorities in Java

Thread priority is a number assigned to a thread that tells the CPU scheduler which thread is more important and should get more chances to execute.

Key Points:

- Priority values range from 1 to 10
- MIN_PRIORITY = 1 → lowest priority
- NORM_PRIORITY = 5 → normal/default priority
- MAX_PRIORITY = 10 → highest priority
- When a thread is created, it gets NORM_PRIORITY (5)

Methods:

- setPriority(int p) → sets thread priority
- getPriority() → returns thread priority

📝 **Note:** Higher priority does NOT guarantee immediate execution. Scheduler gives more chances to higher priority threads — behavior depends on the OS.

Example:-

```
class MyThread extends Thread {
    public void run() {
        System.out.println(Thread.currentThread().getName() +
            " running with priority " + Thread.currentThread().getPriority());
    }
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        t1.setPriority(Thread.MIN_PRIORITY); // 1
        t2.setPriority(Thread.MAX_PRIORITY); // 10
        t1.start();
        t2.start();
    }
}
```

isAlive() and join() Method

Sometimes one thread needs to know when another thread is ending. In Java, isAlive() and join() are two different methods to check whether a thread has finished its execution.

isAlive()

The isAlive() method returns true if the thread upon which it is called is still running; it returns false.

Example:

```
public class MyThread extends Thread {
    public void run() {
        System.out.println("r1 ");
        try {
            Thread.sleep(500);
```

```
        } catch (InterruptedException ie) {}
        System.out.println("r2 ");
    }
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        t1.start();
        t2.start();
        System.out.println(t1.isAlive());
        System.out.println(t2.isAlive());
    }
}
```

Output:

```
r1
true
true
r1
r2
r2
```

Explanation: The t1 thread is created and starts running. CPU gives a chance to t1, so run() is called and prints r1, then it goes to sleep. Meanwhile, t2 starts but has not run yet. The main thread continues and checks if t1 and t2 are alive, so it prints true true. Now the CPU gives a chance to t2, run() is called, and it prints r1. t2 goes to sleep. Then, t1 wakes up and prints r2. At last, t2 wakes up and prints r2.

join() Method

join() method is used more commonly than isAlive(). This method waits until the thread on which it is called terminates. Using the join() method, we tell our thread to wait until the specified thread completes its execution.

Example WITHOUT join():

```
public class MyThread extends Thread {
    public void run() {
        System.out.println("r1 ");
        try {
            Thread.sleep(500);
        } catch (InterruptedException ie) {}
        System.out.println("r2 ");
```

```
    }
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        t1.start();
        t2.start();
    }
}
Output: r1  r1  r2  r2
```

Example WITH join():

```
public class MyThread extends Thread {
    public void run() {
        System.out.println("r1 ");
        try {
            Thread.sleep(500);
        } catch (InterruptedException ie) {}
        System.out.println("r2 ");
    }
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        t1.start();
        try {
            t1.join(); // Waiting for t1 to finish
        } catch (InterruptedException ie) {}
        t2.start();
    }
}
Output: r1  r2  r1  r2
```

Explanation: The join() method on thread t1 ensures that t1 finishes its process before thread t2 starts.

Race Condition in Multithreading

Sometimes multiple threads try to access and modify the same shared data at the same time. When this happens without proper synchronization, the final result becomes unpredictable. This situation is called a **Race Condition**.

A **Race Condition** occurs when two or more threads compete to modify shared resources, and the output depends on which thread executes first.

Race conditions usually occur when:

- Multiple threads access shared variables
- Threads modify data simultaneously
- No synchronization is used

Example:-

```
class Counter {
  int count = 0;
  public void increment() {
     count++; // Not thread-safe
  }

}



public class RaceConditionExample {

  public static void main(String[] args)
        throws InterruptedException {

     Counter counter = new Counter();

     Thread t1 = new Thread(() -> {

        for (int i = 0; i < 1000; i++) {
           counter.increment();
        }

     });
```

```
        Thread t2 = new Thread(() -> {

            for (int i = 0; i < 1000; i++) {
                counter.increment();
            }
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println(counter.count);
    }

}
```

Output (May Vary Every Time)

```
1789
1932
1998
2000
```

Java Thread Synchronization

When we start two or more threads within a program, there may be a situation when multiple threads try to access the same resource, and finally, they can produce unforeseen results due to concurrency issues. For example, if multiple threads try to write within the same file, then they may corrupt the data because one of the threads can override data, or while one thread is opening the same file at the same time, another thread might be closing it.

So there is a need to synchronize the action of multiple threads and make sure that only one thread can access the resource at a given point in time.

Why is Synchronization Needed?

- Prevents Data Inconsistency: Ensures that multiple threads don't corrupt shared data when accessing it simultaneously.

- Avoids Race Conditions: Allows only one thread to execute a critical section at a time, maintaining predictable results.
- Maintains Thread Safety: Protects shared resources from concurrent modification by multiple threads.
- Ensures Data Integrity: Keeps shared data accurate and consistent throughout program execution.

Synchronisation	Static Synchronisation
It needs the use of an object-level lock.	It needs the use of a class-level lock.
It is not necessary to declare its method as static.	Its method must be marked as static.
It is frequently used.	It isn't used on a regular basis.
For each object, a new instance is produced.	For the entire programme, there is only one instance.

Example (Synchronized Static Method):

```
public class MyThread extends Thread {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        t1.start();
        t2.start();
    }
    public void run() {
        printTable(2);
        printTable(10);
    }
    synchronized static void printTable(int n) {
        for (int i = 1; i <= 3; i++) {
            System.out.println(n * i);
            try {
                Thread.sleep(100); // small delay to show sync effect
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
```

```
    }
}
Output: 2  4  6  10  20  30  2  4  6  10  20  30
```

Explanation: t1 thread starts. The run() function is called, and it calls printTable(), which is synchronized (only one thread can enter this method at a time), so even if there are two threads, only one thread executes while the other waits. So t1 prints tables of 2 and 10, then t2 starts and prints tables of 2 and 10.

Synchronized Block in Java Multithreading

Sometimes multiple threads try to access the same resource at the same time, which can cause data inconsistency. In Java, a **synchronized block** is used to control access to a specific part of code so that only one thread can execute that block at a time.

Synchronized Block

A **synchronized block** allows only one thread to execute a block of code at a time using a given object as a lock. It is used when we want to synchronize **only a portion of code** instead of the entire method.

Syntax:

```
synchronized(object) {
  // critical section
}
```

The object inside synchronized is called the **monitor lock.** Only one thread can hold the lock on that object at a time.

Example WITHOUT synchronized block:

```
class MyThread {
    void printNumbers() {
    for (int i = 1; i <= 5; i++) {
        System.out.println(i);
        try {
            Thread.sleep(500);
        } catch (InterruptedException ie) {}
```

```
    }
    }
}
public class Test {
    public static void main(String[] args) {

    MyThread obj = new MyThread();

    Thread t1 = new Thread(() -> obj.printNumbers());
    Thread t2 = new Thread(() -> obj.printNumbers());

    t1.start();
    t2.start();
    }
}
```

Output:

```
1
1
2
2
3
3
4
4
5
5
```

Explanation:

Both t1 and t2 threads execute the printNumbers() method at the same time. Since there is no synchronization, their execution overlaps, and numbers from both threads are printed in mixed order.

Example WITH synchronized block:

```
class MyThread {

    void printNumbers() {
```

```
    synchronized(this) {
        for (int i = 1; i <= 5; i++) {
            System.out.println(i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException ie) {}
        }
    }

    }
}

public class Test {
    public static void main(String[] args) {

    MyThread obj = new MyThread();

    Thread t1 = new Thread(() -> obj.printNumbers());
    Thread t2 = new Thread(() -> obj.printNumbers());

    t1.start();
    t2.start();
    }
}
```

Output:

```
1
2
3
4
5
1
2
3
4
5
```

Explanation:

Thread t1 starts and enters the synchronized block by acquiring the lock on the object. While t1 is executing the synchronized block, t2 must wait until the lock is released. After t1 finishes execution and releases the lock, t2 acquires the lock and executes the block. This ensures that numbers are printed sequentially without mixing.

Advantages of Synchronized Block

- Improves data consistency
- Allows synchronization of only the required code
- Improves performance compared to synchronized methods
- Prevents thread interference

Volatile Keyword in Java

The volatile keyword is used to ensure that the value of a variable is always read from and written to the main memory, not from a thread's local cache.

Problem Without volatile:

Each thread may store a local copy (cache) of a variable. If one thread changes the variable, other threads may still see the old (cached) value — leading to visibility issues.

Solution: Declare the variable as volatile.

```
class SharedData {
    volatile boolean flag = true; // visible to all threads
}

class MyThread extends Thread {
    SharedData data;
    MyThread(SharedData d) { this.data = d; }
    public void run() {
        while (data.flag) {
            // keeps running until flag becomes false
        }
```

```
        System.out.println("Thread stopped");
    }
    public static void main(String[] args) throws InterruptedException {
        SharedData d = new SharedData();
        MyThread t = new MyThread(d);
        t.start();
        Thread.sleep(1000);
        d.flag = false; // visible immediately because of volatile
        System.out.println("Flag set to false");
    }
}
```

📝 **Note:** volatile ensures visibility of changes across threads. However, it does NOT ensure atomicity — for compound operations like i++, use synchronized.

Feature	volatile	synchronized
Visibility	Yes — changes visible to all threads	Yes
Atomicity	No	Yes
Locking	No lock needed	Uses a monitor lock
Use case	Simple flags or state variables	Critical sections with multiple operations

Thread Communication — wait(), notify(), notifyAll()

These methods are used for thread communication. They belong to the Object class, not Thread.

- wait() → Thread goes to sleep and releases the lock
- notify() → Wakes up one waiting thread
- notifyAll() → Wakes up all waiting threads

📝 **Note:** Must be used inside a synchronized block/method. Otherwise → IllegalMonitorStateException at runtime.

Example: Food Ready in Kitchen
Cook (Producer) prepares food. Waiters (Consumers) are waiting.

- If food is not ready → waiters wait()
- When food is ready → cook calls notify() / notifyAll()

```
class Restaurant {
    boolean foodReady = false;
```

```
    synchronized void waitForFood() {
        while (!foodReady) {
            try {
                System.out.println("Waiter is waiting...");
                wait(); // Thread goes to WAITING state and releases lock
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        System.out.println("Waiter got the food!");
    }

    synchronized void prepareFood() {
        System.out.println("Cook is preparing food...");
        foodReady = true;
        notify(); // Wakes up one waiting thread
        System.out.println("Cook notified waiter");
    }
}

public class Test {
    public static void main(String[] args) {
        Restaurant obj = new Restaurant();
        Thread waiter = new Thread(() -> obj.waitForFood());
        Thread cook = new Thread(() -> {
            try { Thread.sleep(2000); } catch (Exception e) {}
            obj.prepareFood();
        });
        waiter.start();
        cook.start();
    }
}
```

What Happens Step-by-Step:

1. Waiter thread starts
2. foodReady = false → calls wait()

3. Waiter releases the lock + goes to the WAITING state
4. Cook prepares food
5. Cook calls notify()
6. The waiter wakes up and continues

notify() vs notifyAll():

Method	Wakes	Use Case
notify()	Only ONE waiting thread	One waiter needed
notifyAll()	ALL waiting threads (they compete for the lock)	Multiple waiters are waiting for food

sleep()

Sometimes a thread needs to pause its execution temporarily or wait for another thread to complete some task. In Java, sleep() and wait() are two different methods used to pause thread execution, but they work differently.

sleep()

The sleep() method causes the currently executing thread to pause for a specified amount of time. During sleep, the thread does not release the lock it holds.

Method Syntax:

Thread.sleep(milliseconds);

It throws InterruptedException, so it must be handled using try-catch.

Example:

```
public class MyThread extends Thread {
   public void run() {
      System.out.println("Start ");
   try {
         Thread.sleep(1000);
   } catch (InterruptedException ie) {}
      System.out.println("End ");
   }
   public static void main(String[] args) {
   MyThread t1 = new MyThread();
   MyThread t2 = new MyThread();
```

```
    t1.start();
    t2.start();
    }
}
Output:
Start
Start
End
End
```

Explanation:

t1 thread starts and prints Start, then it goes to sleep for 1 second. Meanwhile, t2 thread also starts and prints Start, then it goes to sleep. After 1 second, t1 wakes up and prints End, followed by t2 printing End. During sleep, the thread pauses but does not release the lock.

Important: sleep() vs wait()

Feature	sleep()	wait()
Defined in	Thread class	Object class
Releases lock	NO — keeps the lock	YES — releases the lock
Used inside synchronized	Not required	Required
Woken by	Time expiry	notify() / notifyAll() or timeout
Thread state	TIMED_WAITING	WAITING or TIMED_WAITING

Deadlock in Java

A Deadlock is a situation where two or more threads are blocked forever, each waiting for a resource that the other thread holds.

Real-life Example:
Person A has a pen and needs paper. Person B has a paper and needs a pen. Neither gives what they have → both are stuck forever.

Deadlock occurs when ALL four conditions are true (Coffman Conditions):

- Mutual Exclusion: Only one thread can use a resource at a time
- Hold and Wait: Thread holds one resource and waits for another
- No Preemption: Resources cannot be forcibly taken
- Circular Wait: Thread A waits for B, B waits for A

Example (Deadlock):

```
public class DeadlockExample {
    static Object lock1 = new Object();
    static Object lock2 = new Object();

    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            synchronized (lock1) {
                System.out.println("T1: Holding lock1");
                try { Thread.sleep(100); } catch (Exception e) {}
                synchronized (lock2) { // waits for lock2 (held by t2)
                    System.out.println("T1: Holding lock1 and lock2");
                }
            }
        });

        Thread t2 = new Thread(() -> {
            synchronized (lock2) {
                System.out.println("T2: Holding lock2");
                try { Thread.sleep(100); } catch (Exception e) {}
                synchronized (lock1) { // waits for lock1 (held by t1)
                    System.out.println("T2: Holding lock1 and lock2");
                }
            }
        });
        t1.start();
        t2.start();
    }
}
```

How to Avoid Deadlock:

- Always acquire locks in the same order in all threads

- Minimize synchronized blocks — keep them small
- Avoid nested locks where possible

Viva Questions

1. What is wait()?
2. Where is wait() defined?
3. What is the difference between wait() and sleep()?
4. Why do we use wait()?
5. What does notify() do?
6. What does notifyAll() do?
7. Can we call wait() outside a synchronized block?
8. Why must wait() be used inside synchronized?
9. Does notify() release the lock immediately?
10. Why should we use while instead of if with wait()?
11. What happens after notifyAll() is called?
12. Can notify() wake a specific thread?
13. What is thread communication?
14. What happens when multiple threads are waiting?
15. Which exception occurs if wait() is used incorrectly?

MCQs

Q1. Where are wait(), notify(), notifyAll() defined?
A. Thread class
B. Object class
C. System class
D. Runnable interface

Q2. What happens when wait() is called?
A. The thread keeps lock
B. Thread releases the lock and waits
C. Thread terminates
D. Thread runs immediately

Q3. Which method wakes all waiting threads?
A. notify()
B. wakeAll()
C. notifyAll()
D. resumeAll()

Q4. What happens if wait() is called outside a synchronized?
A. Compile-time error
B. Runtime error
C. No error
D. Deadlock

Q5. Which method releases the lock?
A. sleep()
B. wait()
C. Both
D. None

Q6. After calling notify(), a thread:
A. Runs immediately
B. Goes to waiting state
C. Moves to runnable state
D. Terminates

Q7. Which is better when multiple threads are waiting?
A. notify()
B. notifyAll()
C. sleep()
D. yield()

Q8. Which exception is thrown if the synchronization rule is violated?
A. NullPointerException
B. InterruptedException
C. IllegalMonitorStateException
D. IOException

Q9. What is the main purpose of wait() and notify()?
A. Memory management
B. Thread communication
C. File handling
D. Input/output

Q10. wait() must be called inside:
A. Main method
B. Any block
C. Synchronized block
D. Try block

Multithreading – Coding Questions (Coder's Task)

Q1 – Extend Thread

Create a class MyThread that extends Thread and prints "Hello from ThreadX" where X is the thread number. Start 3 threads.

Q2 – Implement Runnable

Create a class MyRunnable implementing Runnable. Start 2 threads using the Thread constructor and print "Runnable ThreadX running".

Q3 – Using sleep()

Write a program where a thread prints numbers 1 to 5, with 1 second pause between each number.

Q4 – Using join()

Create 2 threads: T1 prints 1-5, T2 prints 6-10. Ensure T2 starts printing only after T1 finishes.

Q5 – Using yield()

Create 2 threads that print messages in a loop. Use Thread.yield() inside one thread. Observe and explain which thread gets CPU preference.

Q6 – Synchronized Method

Create a Counter class with an increment() method. Start 2 threads that increment the counter 1000 times each. Print the final value and explain why synchronization is required.

Q7 – Wait & Notify

Create a Restaurant class:

- Cook thread prepares food.
- Waiter thread waits for food using wait().
- Cook calls notify() when food is ready.

- Print in order of events.

Q8 – Thread Priorities

Create 3 threads with different priorities. Observe the order of execution. Print the thread name and priority.

Q9 – Thread Chaining / Reverse Order

Create a program like ReverseGreet where each thread creates the next, up to n threads, printing in reverse order.